SIKSHA 'O' ANUSANDHAN (DEEMED TO BE UNIVERSITY)
Institute of Technical Education and Research | Department of Computer Science & Engineering
CSE 4571 – PRACTICAL ROBOTICS PROJECTS WITH ARDUINO

EXPERIMENT 5
SENSOR-BASED AUTOMATIC ACTUATOR CONTROL SYSTEM

Duration	Level	Mode	CO Mapping
2 Hours	B. Tech CSE Final Year	Hardware / Wokwi	CO2, CO3, CO4

1. Aim of the experiment

To integrate a PIR motion sensor, servo and buzzer to create an automatic robotic response.

2. Objectives

- Read a digital motion event.
- Apply conditional control.
- Move an actuator automatically.
- Generate an event alert.

3. Components / TinkerCAD or Wokwi Parts

Component	Qty.	Purpose
Arduino UNO	1	Microcontroller
PIR sensor	1	Motion-event input
Micro Servo	1	Automatic actuator
Buzzer	1	Event alert

4. Theory

A robot combines sensing, decision-making and actuation. A PIR provides a digital motion event; the Arduino converts that event into an actuator command. In this experiment, motion commands the servo to 90° and activates a 1500 Hz buzzer tone. With no motion, the servo returns to 0° and the buzzer is silenced.

5. Pre-Lab Questions

1. What is a digital sensor?

2. Why is conditional logic central to robotics?

6. Connection Table

Signal	Pin	Function
PIR OUT	D2	Digital motion input
Servo signal	D9	Servo command
Buzzer	D6	Alert output
VCC	5V	Supply
GND	GND	Common ground

7. Circuit Diagram for Implementation

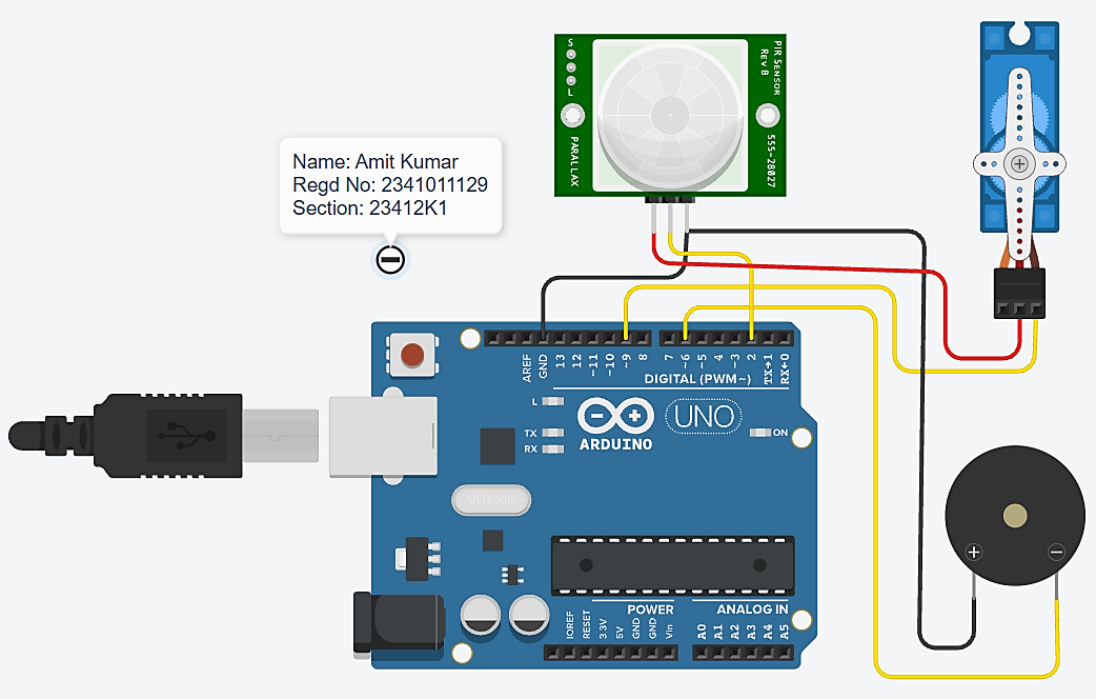


Figure 1. Verified PIR–servo–buzzer automatic response circuit.

8. Circuit Diagram Implemented Using Hardware's

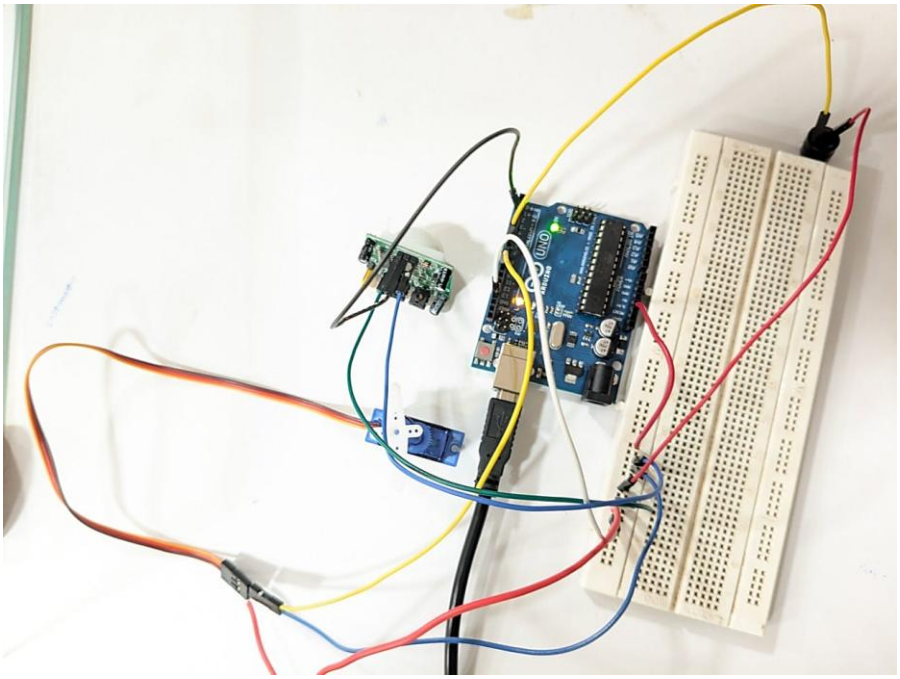


Figure 2. Hardware Implementation and Verified PIR–servo–buzzer automatic response circuit.

9. Procedure

- Build the circuit.
- Run the program.
- Trigger motion in Wokwi.
- Observe servo and buzzer response.
- Repeat and record motion/no-motion behavior.

10. Arduino Program

```
#include <Servo.h>
const int PIR_PIN = 2;
const int SERVO_PIN = 9;
const int BUZZER_PIN = 6;
const int SAFE_ANGLE = 0;
const int MOTION_ANGLE = 90;
const int BUZZER_FREQUENCY = 1500;
Servo gateServo;
int previousMotionState = -1;
void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  gateServo.attach(SERVO_PIN);
  gateServo.write(SAFE_ANGLE);
  noTone(BUZZER_PIN);
  Serial.begin(9600);
  Serial.println("Experiment 5: Automatic Actuator Control");
}
void loop() {
  int motionState = digitalRead(PIR_PIN);
  if (motionState != previousMotionState) {
    if (motionState == HIGH) {
      Serial.println("MOTION");
      gateServo.write(MOTION_ANGLE);
      tone(BUZZER_PIN, BUZZER_FREQUENCY);
    } else {
      Serial.println("SAFE");
      gateServo.write(SAFE_ANGLE);
      noTone(BUZZER_PIN);
    }
    previousMotionState = motionState;
  }
  delay(20);
}
```

11. Observation Table

Sl. No.	Motion State	Servo Angle	Buzzer State
1	Motion detected	90°	ON — 1500 Hz
2	No motion	0°	OFF (Silent)
3	Motion detected	90°	ON — 1500 Hz
4	No motion	0°	OFF (Silent)
5	Motion detected	90°	ON — 1500 Hz

12. Expected Output / Analysis

Motion: Serial Monitor shows MOTION, servo moves to 90°, and the buzzer sounds at 1500 Hz.

No motion: SAFE is displayed, the servo returns to 0°, and the buzzer is silenced.

13. Result

The PIR sensor, micro servo and buzzer were successfully integrated to demonstrate automatic sensing, conditional decision-making and actuator/alert response in Wokwi.

14. Post-Lab Questions

3. Keep the gate open for 3 seconds after motion.

4. Add a second sensor and define an AND condition.

5. What is event-driven control?

15. Precautions

- Verify pin assignments before running the simulation.
- Use common ground for connected devices.
- Record both input condition and observed output.
- For physical implementation, consider current, voltage, mechanical loading and sensor tolerances.

16. Suggested Extension

Modify the program to add one meaningful feature, document the change, and preserve the original behavior as a baseline.

```
#include <Servo.h>
// CSE 4571 - Experiment 5
// Extension: Keep gate open for 3 seconds after motion
const int PIR_PIN = 2;
const int SERVO_PIN = 9;
const int BUZZER_PIN = 6;
const int SAFE_ANGLE = 0;
const int OPEN_ANGLE = 90;
const int BUZZER_FREQUENCY = 1500;
const unsigned long GATE_HOLD_TIME = 3000;
Servo gateServo;
int previousMotionState = -1;
unsigned long lastMotionTime = 0;
bool gateOpen = false;
void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  gateServo.attach(SERVO_PIN);
  gateServo.write(SAFE_ANGLE);
  noTone(BUZZER_PIN);
  Serial.begin(9600);
  Serial.println("Experiment 5: 3-Second Gate Hold");
}
void loop() {
  int motionState = digitalRead(PIR_PIN);
  unsigned long currentTime = millis();
  // Update the motion event
  if (motionState == HIGH) {
    lastMotionTime = currentTime;
    if (!gateOpen) {
      gateServo.write(OPEN_ANGLE);
      gateOpen = true;
    }
    tone(BUZZER_PIN, BUZZER_FREQUENCY);
  } else {
    noTone(BUZZER_PIN);
  }
  // Print only when the sensor state changes
  if (motionState != previousMotionState) {
    if (motionState == HIGH) {
      Serial.println("MOTION");
    } else {
      Serial.println("SAFE");
    }
    previousMotionState = motionState;
  }
  // Close the gate 3 seconds after the last detected motion
  if (gateOpen &&
      motionState == LOW &&
      currentTime - lastMotionTime >= GATE_HOLD_TIME) {
    gateServo.write(SAFE_ANGLE);
    gateOpen = false;
    Serial.println("Gate closed after 3 seconds");
  }
}
```

Suggested Extension: Three-Second Gate Hold

Feature added: The program was modified to keep the servo-operated gate open for three seconds after the PIR sensor stops detecting motion.

Implementation: The Arduino millis() function was used to track elapsed time without blocking sensor monitoring. The gate closes when the PIR output is LOW and three seconds have elapsed since the last detected motion.

Preserved baseline behavior: The original pin assignments remain unchanged: PIR OUT on D2, servo signal on D9, and buzzer on D6. Motion detection still opens the gate to 90° and activates the 1500 Hz buzzer. When motion is absent, the buzzer is silenced. The gate returns to 0° after the three-second hold period.

Expected outcome: The gate remains open briefly after motion ends, providing a delayed-closing mechanism while retaining the original automatic sensing and alert functions

17. Student Record

Amit Kumar	2341011129		
Student Name	**Registration No.**	**Date**	**Faculty Signature**